



Universidade do Minho
Licenciatura em Ciências da Computação

Programação Concorrente

2025/2026

Mini-Jogo Concorrente Multijogador

Relatório de Projeto — Grupo 17

Hugo Gomes [100056] • Cláudio Ferreira [108577]

Marco Abreu [108578] • Nelson Sousa [109068]

Maio 2026

1. Introdução

Este relatório descreve o desenvolvimento de um mini-jogo multijogador concorrente, realizado no âmbito da unidade curricular de Programação Concorrente (PC) do ano letivo 2025/2026, pelo Grupo 17.

O objetivo do projeto foi conceber e implementar um sistema distribuído cliente-servidor capaz de suportar múltiplos jogadores em simultâneo, aplicando os princípios de concorrência, sincronização e comunicação por rede estudados ao longo da unidade curricular.

O sistema foi desenvolvido em três componentes tecnológicas distintas: um servidor em Erlang, um cliente gráfico em Processing e uma biblioteca de comunicação de rede em Java. A integração destas três tecnologias constitui o cerne do trabalho e demonstra a aplicação prática de conceitos de programação concorrente num contexto de jogo em tempo real.

2. Arquitetura do Sistema

O sistema segue uma arquitetura cliente-servidor clássica, onde o servidor centraliza toda a lógica de jogo e os clientes são responsáveis pela apresentação gráfica e captura de input do utilizador.

2.1 Visão Geral

A comunicação entre cliente e servidor é feita através de TCP/IP, com o servidor a escutar na porta 12345 (localhost).

Componente	Tecnologia	Responsabilidade
Servidor	Erlang	Lógica de jogo, autenticação, fila, sessões, scoreboard
Cliente Gráfico	Processing	Interface visual, input do utilizador, ecrã de login
Biblioteca de Rede	Java	Comunicação TCP, gestão de estado, protocolo de mensagens

2.2 Fluxo de uma Sessão

- O jogador abre o cliente Processing e vê o ecrã de login.
- Autentica-se com as suas credenciais via TCP para o servidor Erlang.
- Entra na fila de espera (queue_manager) até haver jogadores suficientes para uma partida.
- O game_supervisor cria uma nova game_session para o grupo de jogadores.
- Decorre a partida com envio contínuo de eventos de teclado (esquerda/direita/frente).
- No fim da partida, o scoreboard_manager regista a melhor pontuação de cada jogador.
- O jogador pode sair da fila (leave_queue) ou fazer logout.

3. Servidor Erlang

O servidor constitui o núcleo do sistema e foi implementado integralmente em Erlang, tirando partido do modelo de atores (Actor Model) da plataforma BEAM para gerir concorrência de forma natural e segura.

3.1 Módulos Implementados

Módulo	Função
server.erl	Ponto de entrada; inicializa o socket TCP e aceita conexões
connection_session.erl	Gere cada ligação TCP individual como um processo Erlang independente
auth_manager.erl	Regista e valida credenciais dos jogadores
queue_manager.erl	Mantém a fila de espera; dispara partidas quando há jogadores suficientes
game_supervisor.erl	Supervisiona as partidas ativas; relança sessões em caso de falha
game_session.erl	Representa uma partida em curso; coordena os jogadores envolvidos
game_engine.erl	Motor do jogo: física, deteção de capturas, atualização de estado
scoreboard_manager.erl	Persiste e consulta as melhores pontuações por jogador

3.2 Concorrência e Supervisão

Cada ligação TCP é tratada por um processo Erlang isolado (`connection_session`), garantindo que a falha de um cliente não afeta os restantes. O `game_supervisor` implementa uma estratégia de supervisão que permite reiniciar sessões de jogo em caso de erro, aumentando a robustez do sistema.

O modelo de passagem de mensagens assíncronas do Erlang foi utilizado para coordenar os diferentes módulos sem recorrer a locks ou mutexes explícitos, evitando condições de corrida de forma estrutural.

4. Cliente: Java + Processing

O cliente é composto por duas camadas: uma biblioteca Java responsável pela comunicação de rede e gestão de estado, e um ambiente Processing que fornece a interface gráfica ao utilizador.

4.1 Biblioteca Java (client.jar)

O código fonte Java encontra-se em `src/cliente_java/src/` e é compilado para `client.jar`. Os principais pacotes são:

- `network` — `ServerConnection`: gere a ligação TCP e implementa o protocolo de mensagens definido. `MessageProtocol` define os tipos de mensagem trocados com o servidor Erlang.
- `input` — `InputHandler`: captura eventos de teclado e traduz para comandos de protocolo (esquerda, direita, frente).
- `state` — `GameState`, `GamePhase`, `ObjectType`: representam o estado do jogo localmente para suportar a renderização em Processing.
- `mock` — `MockServer`: servidor falso para testar a comunicação Java sem necessidade do servidor Erlang.

4.2 Cliente Gráfico Processing

O cliente Processing (`src/cliente_processing/MiniJogo/`) utiliza a biblioteca `client.jar` para comunicar com o servidor e apresenta ao utilizador:

- Ecrã de login com campos de utilizador e palavra-passe (com zonas clicáveis corrigidas na versão final).
- Sala de espera enquanto aguarda outros jogadores na fila.
- Ambiente de jogo com visualização em tempo real das posições e pontuações dos jogadores.

O cliente foi testado com 3 a 4 instâncias simultâneas para formar uma partida completa.

5. Protocolo TCP

A comunicação entre o cliente Java e o servidor Erlang segue um protocolo de mensagens de texto definido pelo grupo.

5.1 Principais Mensagens

Direção	Mensagem	Descrição
Cliente → Servidor	<code>login <user> <pass></code>	Autenticação do jogador
Cliente → Servidor	<code>join_queue</code>	Entrar na fila de espera
Cliente → Servidor	<code>leave_queue</code>	Sair da fila de espera
Cliente → Servidor	<code>move <esquerda direita frente></code>	Enviar input de movimento
Cliente → Servidor	<code>logout</code>	Terminar sessão
Servidor → Cliente	<code>game_state <dados></code>	Atualização do estado do jogo

Servidor → Cliente	game_over <pontuação>	Fim da partida com resultado
--------------------	-----------------------	------------------------------

Um aspeto relevante do protocolo é a tradução das teclas: o cliente Java converte os eventos de teclado em comandos textuais (esquerda/direita/frente) que o servidor Erlang interpreta diretamente, sem necessidade de mapeamento adicional no lado do servidor.

6. Dificuldades e Soluções

Durante o desenvolvimento foram identificados e resolvidos vários problemas técnicos, documentados nas alterações finais:

Problema	Solução Aplicada
Configuração TCP incorreta do socket aceite	Socket fica ativo apenas após passagem para <code>connection_session</code>
Tradução incorreta das teclas do protocolo	Corrigido mapeamento para left/right/forward → esquerda/direita/frente
Física: velocidade parava ao soltar tecla	Mantida velocidade linear após deixar de premir a tecla frente
Scoreboard guardava pontuação errada	Corrigido para guardar melhor pontuação vencedora baseada em capturas
Zonas clicáveis do ecrã de login incorretas	Recalculadas as coordenadas dos botões e campos de texto
Ficheiros duplicados e crash dumps na entrega	Criada estrutura limpa de entrega sem artefactos desnecessários
Fluxo <code>leave_queue/logout</code> incorreto	Assegurado envio de <code>leave_queue</code> antes de <code>logout</code> ao sair da sala de espera

7. Conclusão

O projeto cumpriu os objetivos propostos: foi implementado um mini-jogo multijogador concorrente funcional, com servidor em Erlang, cliente gráfico em Processing e biblioteca de comunicação em Java.

A utilização de Erlang para o servidor revelou-se uma escolha adequada para o problema em questão, pois o modelo de atores e a tolerância a falhas integrada na plataforma BEAM facilitaram a implementação de concorrência segura sem os problemas típicos de condições de corrida ou deadlocks.

A separação entre a lógica de rede (Java) e a apresentação gráfica (Processing) permitiu um desenvolvimento modular e facilitou o teste independente dos componentes, como demonstrado pelo MockServer.

Em termos de trabalho futuro, seria interessante explorar a persistência de dados no scoreboard entre sessões do servidor, adicionar salas de jogo configuráveis e melhorar a interface gráfica do cliente Processing.

Referências

- Documentação Erlang/OTP — <https://www.erlang.org/doc/>
- Processing Foundation — <https://processing.org/reference/>
- Java SE Documentation — <https://docs.oracle.com/en/java/>